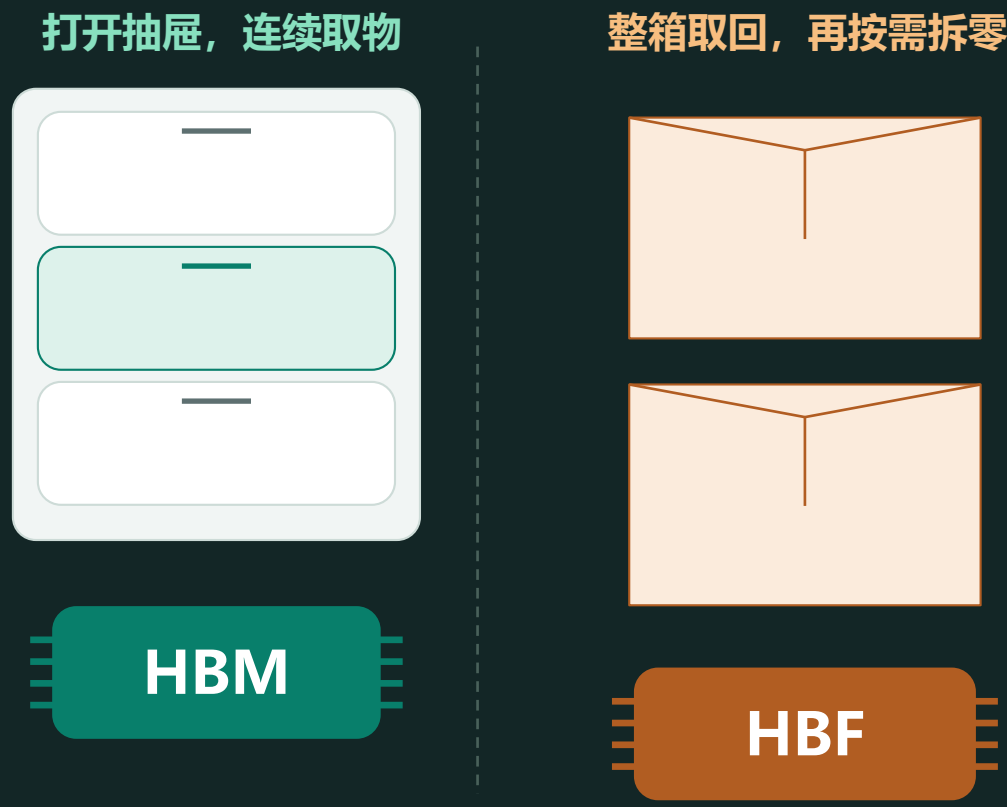
HBM 与 HBF
仿真器究竟模拟到哪一层

从地址、介质动作与共享资源，
解释一次访问如何影响 Token 时间

代码驱动的机制讲解

技术细节 × 日常比喻 × 可编辑示意图

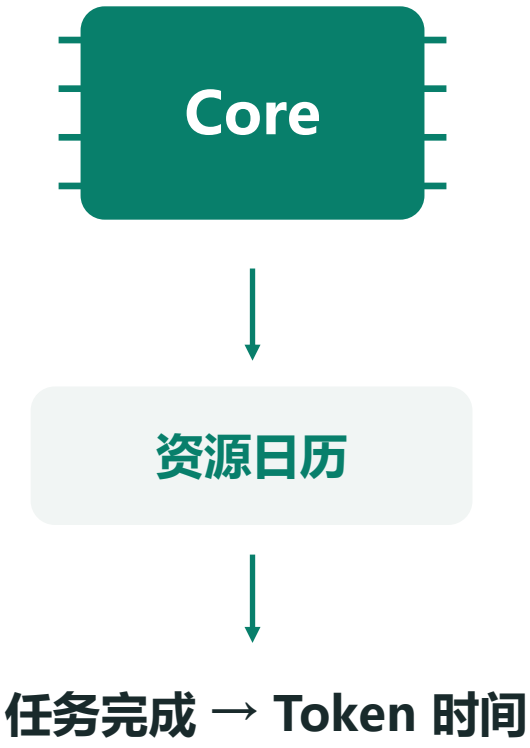


基于 2026-10-26 工作区代码；参数示例不代表实测芯片

五个观察层级

沿层级向下看细节，再沿执行结果向上解释性能

1	**组织与参数**	有哪些单元？容量、几何、带宽与时序
2	**数据与地址**	数据放在哪里？owner、allocation、offset
3	**介质状态与动作**	要做什么？开行、读页、编程、缓存替换
4	**共享资源与事件**	何时能做？准入、排队、依赖与资源日历
5	**推理性能输出**	造成什么影响？算子、Token、TTFT / TPOT



代码：physical_contract.py · data_motion.py · event_kernel.py · reporting.py

组织坐标、状态、执行资源

可被寻址的一层，不一定拥有独立总线或独立队列

1 组织坐标

它在哪里

Bank / Row
Plane / Block / Page

像楼栋、房间与抽屉编号

2 介质状态

它现在怎样

打开哪一行
缓存哪一页、何时就绪

像抽屉是否打开、箱子是否取回

3 执行资源

谁能同时工作

命令总线 / DQ
阵列 / 内部通道 / Host

像共用柜台、走廊与出口

代码：memory_mapping.py · _hbm_control.py · nand_core.py

共同入口：张量怎样变成一次访问

数据身份先绑定分配区间，真正派发任务时再提交给内存 Core



仓位比喻

货物编号 → 仓位起点 → 第几格 → 本次取多少

访问范围与地址校验

访问不能越出 allocation
显式地址须与分配绑定一致，
并落在设备容量内

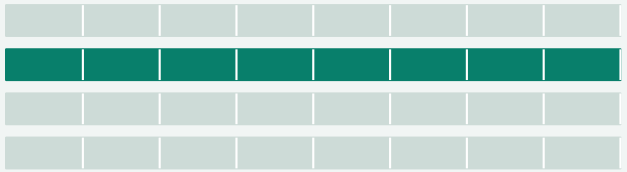
代码：planner.py · memory_allocator.py · data_motion.py · memory_mapping.py

HBM：从 Stack 寻址到 Burst Column

示例：16H、32 Gb Die 的 HBM4 标准参考组织；不是已知厂商器件配置

Stack	→	Channel	→	PC	→	Bank Group	→	Bank
1 堆		32 / Stack		2 / Channel		8 / PC		8 / Group

Bank 内部继续寻址



Row → 32 B Burst Column

容量不能重复乘 Die

64 PC × 64 Bank × 16,384 Row × 1,024 B = 64 GiB / Stack

per_stack 几何已经包含 16H；
SID 是从 Bank 序号派生的标签

代码：hbm_geometry.py · memory_mapping.py · 测试 test_hbm_stack_geometry.py

HBM：两个 PC，共享命令、各走数据

以下按 jedec_hbm 模式说明；命令能否发出与数据通路是否空闲是两件事

一个物理 Channel

共享 Row CA
ACT / PRE / REF

共享 Column CA
RD / WR

两类命令窗口都服务 PC0 与 PC1

PC0 • 各 Bank 状态

独立 DQ 数据通路

PC1 • 各 Bank 状态

独立 DQ 数据通路

Bank 开行与时序状态、命令和 DQ 分开维护

两个柜台窗口

同一类命令共用窗口；
不能因为 PC 不同就同时发出

两条传送带

PC0 与 PC1 各有 DQ；
数据传输可以独立排期

代码：dram_core.py:98–102 · _hbm_control.py

HBM：同样是读，行状态会改变动作

比喻：已开的抽屉直接取；关着的先开；开错的先关再开

行命中

目标激活页已打开



未开行

先打开目标激活页



行冲突

先关闭旧页再打开新页

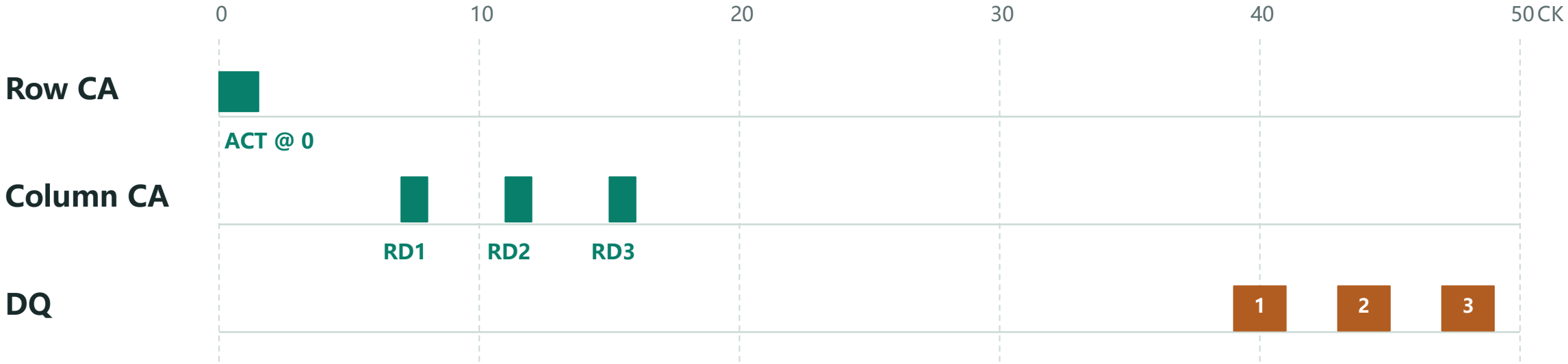


示意不按时间比例；还受 tRAS / tRC、组间间隔、方向切换与可选刷新等约束

HBM：三次同地址读的时钟例子

代码测试：HBM3E 参考、512 GiB 可见容量、1,600 GB/s 接口预算；非实测时延

tCK = 2.56 ns | RL = 32 CK | tCCD = 4 CK | Burst = 2 CK



RD：CK 7 / 11 / 15

DQ：39–41 / 43–45 / 47–49

完成：104.96 / 115.20 / 125.44 ns

来源：tests/test_hbm_clock_arithmetic.py:16–38；ACT 后阵列时序锚点为 CK1

HBF：地址到页，资源分到阵列与通道

通用 NAND 坐标完整保留；论文原生配置把实际 Die 映成 Core Channel

通用坐标 Channel → Target → Die → LUN → Plane → Block → Page → offset

Stack 分组	→	实际 Die	→	Plane	→	Block → Page
16 个实际 Die / Stack		1 个 Core Channel		25 个独立阵列 / Die		擦除单位 → 4 KiB 页

同一实际 Die 的 Plane 共享内部传输通道

Plane 阵列独立 ≠ 内部通道与 Host 出口独立

论文 HBF 示例

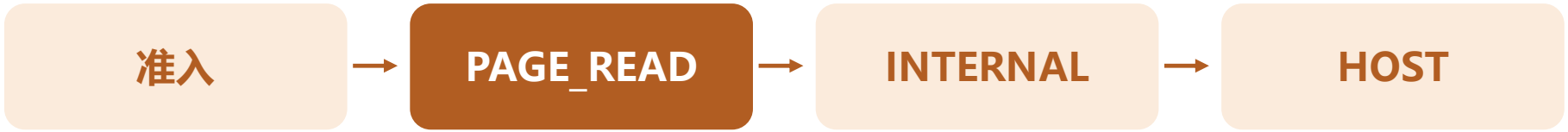
每堆 16 × 25 = 400 个阵列单元
Target / Core Die / LUN 各取 1

代码：memory_mapping.py:124–161 · scripts/exploring_hbf_paper_config.py:272–342

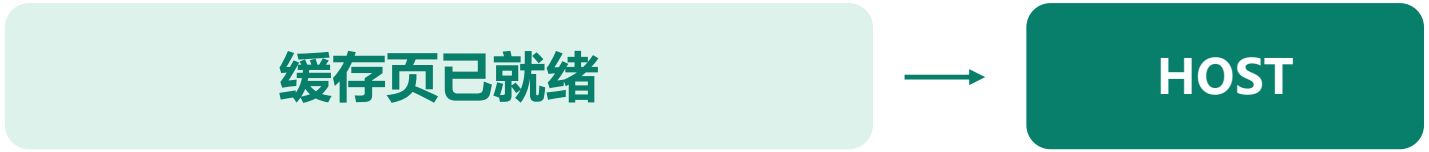
HBF：冷读与命中走不同的路径

整箱拆零：冷读先取回一整页，命中后可复用已取回的页

冷读 / Miss



缓存命中 / Hit



命中仍需等待缓存页就绪与 Host 资源；
缓存替换另受未完成读者约束，避免过早覆盖

代码：nand_core.py:1800–1955 · memory_mapping.py:240–257

HBF：双页缓存与中间 SRAM 是两种模型

一个管理媒体页的复用与覆盖时机；另一个承接显式工作区访问

每阵列的两个读缓存槽

Slot A

page tag · ready
reader_end · LRU

Slot B

page tag · ready
reader_end · LRU

覆盖前必须保护仍在读的页；
两槽支持流水，但不会消除共享通道瓶颈



独立中间 SRAM

显式容量、端口与带宽访问
由 allocator 管理工作区生命周期

代码：nand_core.py:641–676 · sram_core.py · scripts/exploring_hbf_paper_config.py

HBF：媒体写入与可选管理层

写完成的含义，取决于协议模式和是否启用 HOST 暂存

通用 NAND 核心

小写入 → RMW 或拒绝

未启用 write_buffer：
部分页写按配置补读旧页再 Program，
或直接拒绝

可选 write_buffer：有限容量、页合并；
Host 到达可先确认，媒体随后 drain

Strict OCP + HOST 暂存



16 × 256 B = 4 KiB

HOST 页合并 → 整页 Program

上层小型日志式 FTL：异地更新、映射、GC
核心普通写必须整页，完成到媒体；
不能借用 generic 的提前确认语义

代码：nand_core.py · hbf_staging.py · hbf_staging_runtime.py · 测试 test_hbf_staging.py:303–319

跨设备搬运与算子完成时间

媒体服务结果回到任务图，计算、链路与内存的重叠由模型和依赖决定

源端访问 → 链路 / Fabric → 目的端访问

具体任务路径决定串行、分块或流水关系；每个 owner 保留自己的日历与计费

Native kernel 的服务包络



Tkernel = max（其他资源跨度，内存 Core 跨度）+ dependency

公式适用于代码中的 native kernel envelope；不是所有搬运阶段的通用公式

64 B 读请求，为什么可能触发 4 KiB 页读

固定示例：HBM 为 32 B burst；HBF 为 strict OCP 4 KiB 页、64 B Host 粒度

访问条件	介质 / 内部动作	接口有效传输
HBM • 对齐 64 B	2 × 32 B DQ	64 B
HBF • 冷缓存	4 KiB 阵列页 + 内部搬运	64 B Host
HBF • 已就绪命中	跳过阵列读与内部搬运	64 B Host

64×

只比较冷读的媒体页容量与有效载荷；不是延迟倍数，也不是 Host 传输倍数

代码：memory_mapping.py:173–188,240–257 · nand_core.py:1898–1955

怎样把访存影响读成系统性能

一次 transaction 的延迟先影响关键路径，再影响 Token；它不直接等于 TPOT

1 字节账

要多少 / 搬多少

Logical bytes：应用有效载荷

Physical：按 burst / page 计费

2 时间账

跨度 / 忙时 / 总耗时

service_ns：任务 / owner
完成跨度之和（可重叠）

busy：资源实际占用
makespan：总跨度

3 Token 账

TTFT / TPOT

首 Token 时间与后续提交时刻

TPOT 由可见 Token 间隔计算

TPOT =（末个 Token 时间 − 首个 Token 时间）/（可见 Token 数 − 1）

代码：memory_service.py:26–84 · memory_types.py:890–916 · serving.py:17035–17039

模型精度、执行加速、采样预测

这三种开关改变的东西不同，解读结果时必须分开

1 模型选择	HBM analytical / jedec_hbm HBF generic / strict OCP	改变状态、命令、约束与完成点
2 同模型执行加速	少记明细、数值 / 编译路径 可证明分支的 max-plus 跳步	保留该模型的资源竞争与状态转移
3 采样与预测	选中层独立副本执行 fast window / stationary 预测	预测窗口不推进全部 live 物理日历

代码：dram_core.py · _compiled_nand.py · _nand_acceleration.py · native_*sampling / fast_window

已实现的建模深度与边界

准确名称：有状态、物理地址驱动的事务 / 命令与资源时序仿真

范围	HBM	HBF
地址与组织	Stack / Channel / PC / Bank / Row	Channel / LUN / Plane / Block / Page
状态与时序	正常操作命令子集、开行、方向与刷新约束	页读 / Program / Erase；可配置读缓存
可选管理	简化 analytical 或 jedec_hbm；有限请求调度	generic write buffer；strict OCP HOST FTL / GC
未覆盖 / 不宣称	完整 JEDEC、厂商映射、训练与电路行为	完整 AXI、ECC / retention 与 cell 内容行为

机制已经实现 ≠ 参数已对目标芯片校准；研究假设、标准参考与实测证据应分别说明

代码：hbm_geometry.py:379–391 · nand_core.py:353–362 · hbf_staging.py:476–526

理解这个仿真器的三个抓手

先找到数据位置，再看介质动作，最后看谁在等待谁

1

位置

地址层级告诉你
数据落在哪个物理单元

2

动作

HBM 看开行与命令
HBF 看页、缓存与编程

3

竞争

共享总线与任务依赖
共同决定完成时间

HBM 像按抽屉取物；HBF 像整箱拆零。
仿真器真正计算的是：这些动作如何占用资源，并推迟后续任务。

阅读每次实验结果时，同时确认：物理配置、协议模式、是否采用采样预测

附录：按问题阅读代码

每页讲稿备注还包含对应文件和行号，可用于继续核对细节

1	**定义哪些层级?**	memory_types.py hbm_geometry.py / _hbm_dependencies.py	配置与标准组织
2	**地址怎样落到资源?**	memory_mapping.py memory_allocator.py	映射、拆分与分配
3	**访问怎样完成?**	dram_core.py / _hbm_control.py nand_core.py / hbf_staging.py	状态、时序与管理
4	**怎样影响 Token?**	data_motion.py / event_kernel.py memory_service.py / serving.py	回写、依赖与指标

数值例来源：test_hbm_clock_arithmetic.py · test_hbm_stack_geometry.py · test_hbf_staging.py

以上文件位于 src/heterollm_sim；测试位于 tests；论文配置入口位于 scripts